QUESTIONS

Q1 Resolving Multiple-Inheritance Ambiguity
10 marks

Q2 Virtual Base Class for Battery Data
10 marks

Q3 Polymorphism for Renewable Energy
10 marks

Q4 Runtime Polymorphism in Sustainability Dashboards
10 marks

Q5 Dynamic Objects in Smart Grids
10 marks

5/5 answered

Q1 Resolving Multiple-Inheritance Ambiguity

10 marks

A sustainable manufacturing system uses multiple inheritance where two parent classes contain the same calculateImpact() function. Analyze the ambiguity and resolve it using scope resolution.

Your Code

```cpp
#include <iostream>
using namespace std;

class Production {
public:
    void calculateImpact() {
        cout << "Production impact" << endl;
    }
};

class Environment {
public:
    void calculateImpact() {
        cout << "Environmental impact" << endl;
    }
};

class Sustainable : public Production, public Environment {
public:
    void show() {
        Production::calculateImpact();
        Environment::calculateImpact();
    }
};

int main() {
    Sustainable s;
    s.show();

    return 0;
}
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

- **Q1** Resolving Multiple-Inheritance Ambiguity — 10 marks
- **Q2** Virtual Base Class for Battery Data — 10 marks
- **Q3** Polymorphism for Renewable Energy — 10 marks
- **Q4** Runtime Polymorphism in Sustainability Dashboards — 10 marks
- **Q5** Dynamic Objects in Smart Grids — 10 marks

5/5 answered

Q2 Virtual Base Class for Battery Data

10 marks

An EV battery-management system inherits common battery information through multiple classes, causing duplicate data. Analyze the issue and resolve it using a virtual base class.

Your Code

```cpp
#include <iostream>
using namespace std;

class Battery {
protected:
    int capacity;
public:
    Battery(int c) : capacity(c) {}
};

class BatteryA : virtual public Battery {
public:
    BatteryA(int c) : Battery(c) {}
};

class BatteryB : virtual public Battery {
public:
    BatteryB(int c) : Battery(c) {}
};

class EV : public BatteryA, public BatteryB {
public:
    EV(int c) : Battery(c), BatteryA(c), BatteryB(c) {}

    void show() {
        cout << "Battery Capacity: " << capacity << " kWh";
    }
};

int main() {
    EV e(100);
    e.show();
    return 0;
}
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

CO4_AT1_SIMATS_Compiler

View Only

← Back

QUESTIONS

- Q1 — Resolving Multiple Inheritance Ambiguity — 10 marks
- Q2 — Virtual Base Class for Battery Data — 10 marks
- Q3 — Polymorphism for Renewable Energy — 10 marks
- Q4 — Runtime Polymorphism in Sustainability Dashboards — 10 marks
- Q5 — Dynamic Objects in Smart Grids — 10 marks

5/5 answered

Q3 Polymorphism for Renewable Energy

10 marks

A renewable-energy platform calculates energy output from Solar, Wind and Hydro sources. Analyze how abstract classes and runtime polymorphism can provide a flexible solution.

Your Code

```
#include <iostream>
using namespace std;

class Energy {
public:
    virtual void calculate() = 0;
    virtual ~Energy() {}
};

class Solar : public Energy {
public:
    void calculate() {
        cout << "Solar Energy: 100 units" << endl;
    }
};

class Wind : public Energy {
public:
    void calculate() {
        cout << "Wind Energy: 80 units" << endl;
    }
};

class Hydro : public Energy {
public:
    void calculate() {
        cout << "Hydro Energy: 120 units" << endl;
    }
};

int main() {
    Energy *p;
    int ch;

    cout << "1. Solar\n2. Wind\n3. Hydro\n";
    cin >> ch;

    if(ch == 1)
        p = new Solar();
    else if(ch == 2)
        p = new Wind();
    else
        p = new Hydro();

    p->calculate();
    delete p;

    return 0;
}
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

View Only

QUESTIONS

Q1
Resolving Multiple Inheritance Ambiguity
10 marks

Q2
Virtual Base Class for Battery Data
10 marks

Q3
Polymorphism for Renewable Energy
10 marks

Q4
Runtime Polymorphism in Sustainability Dashboards
10 marks

Q5
Dynamic Objects in Smart Grids
10 marks

5/5 answered

Q4 Runtime Polymorphism in Sustainability Dashboards

10 marks

A sustainability dashboard processes SolarPanel, WindTurbine and Battery objects using a common base-class pointer. Analyze how runtime polymorphism supports different energy calculations.

Your Code

```cpp
#include <iostream>
using namespace std;

class Energy {
public:
    virtual void calculate() = 0;
    virtual ~Energy() {}
};

class SolarPanel : public Energy {
public:
    void calculate() override {
        cout << "Solar Energy: 100 units" << endl;
    }
};

class WindTurbine : public Energy {
public:
    void calculate() override {
        cout << "Wind Energy: 80 units" << endl;
    }
};

class Battery : public Energy {
public:
    void calculate() override {
        cout << "Battery Energy: 60 units" << endl;
    }
};

int main() {
    Energy *p;
    int ch;

    cout << "1. Solar\n2. Wind\n3. Battery\n";
    cin >> ch;

    if (ch == 1)
        p = new SolarPanel();
    else if (ch == 2)
        p = new WindTurbine();
    else
        p = new Battery();

    p->calculate();
    delete p;

    return 0;
}
```

Input

stdin input

Output

Visible Test Cases

Test Case 1

View Only

QUESTIONS

Q1 ✓
Resolving Multiple-Inheritance Ambiguity
10 marks

Q2 ✓
Virtual Base Class for Battery Data
10 marks

Q3 ✓
Polymorphism for Renewable Energy
10 marks

Q4 ✓
Runtime Polymorphism in Sustainability Dashboards
10 marks

Q5 ✓
Dynamic Objects in Smart Grids
10 marks

5/5 answered

Q5 Dynamic Objects in Smart Grids

10 marks

A smart-grid application dynamically creates energy-storage objects according to demand. Analyze how pointers can be used for dynamic object creation.

Your Code

```cpp
#include <iostream>
using namespace std;

class EnergyStorage {
public:
    virtual void show() {
        cout << "Energy Storage created" << endl;
    }
    virtual ~EnergyStorage() {}
};

class Battery : public EnergyStorage {
public:
    void show() override {
        cout << "Battery Storage created" << endl;
    }
};

class SolarStorage : public EnergyStorage {
public:
    void show() override {
        cout << "Solar Storage created" << endl;
    }
};

int main() {
    int choice;
    EnergyStorage *p;

    cout << "1. Battery\n2. Solar Storage\n";
    cin >> choice;

    if (choice == 1)
        p = new Battery();
    else
        p = new SolarStorage();

    p->show();

    delete p;
    return 0;
}
```

Input

1

Output

Visible Test Cases

Test Case 1